

Nanny Platform — Phased Development Plan

2026-09-23 · @Someone

Overview

The build is split into 9 phases (Phase 0 to Phase 8). Each phase ends with something working and testable, so you can hand one phase at a time to Claude Code, verify it, commit, and move on.

The platform connects families in Angola with verified nannies. It is not an open marketplace: the platform is the official employer/mediator. It runs matching and interviews, issues contracts, collects payment from families and pays nannies.

Role	What they do on the platform
Nanny (Babá)	Registers, builds a profile, uploads documents, accepts interview invitations, signs contracts, sees payments
Family (Família)	Registers, searches and favorites nannies, submits requests, approves the recommended nanny, signs contract, pays
Admin (mediator)	Verifies nannies, matches requests, runs interviews, creates contracts, records payments, handles disputes

The phase order follows dependencies: foundation, then accounts, then the data each role creates, then the admin workflow that connects them, then money, then communication and launch polish.

```
flowchart LR
    P0[Phase 0<br/>Foundation] --> P1[Phase 1<br/>Auth]
    P1 --> P2[Phase 2<br/>Nanny profile]
    P1 --> P3[Phase 3<br/>Family search]
    P2 --> P4[Phase 4<br/>Admin + matching]
    P3 --> P4
    P4 --> P5[Phase 5<br/>Contracts]
    P5 --> P6[Phase 6<br/>Payments]
    P6 --> P7[Phase 7<br/>Notifications]
    P7 --> P8[Phase 8<br/>Launch]
```

Phases 2 and 3 are independent of each other and can be built in either order.

Color palette

The palette is warm and soft: cream backgrounds, blush sections, dusty coral for actions, dark warm brown for text. Define these once as design tokens in Phase 0 and never hard-code hex values in components.

Token	Element	HEX	Tone	Use it for
<code>bg</code>	Main background	<code>#FDF6EF</code>	Warm cream	Page background, cards on pink sections
<code>bg-pink</code>	Pink section background	<code>#F9E1DD</code>	Soft blush pink	Hero, alternating sections, selected states
<code>primary</code>	Primary button / accent	<code>#E58F89</code>	Dusty coral	Primary buttons, links, active tabs, focus ring
<code>text</code>	Main heading / text	<code>#3F312D</code>	Dark warm brown	Headings, body text, table text
<code>text-muted</code>	Secondary text	<code>#826758</code>	Warm muted brown	Labels, helper text, timestamps, placeholders
<code>logo</code>	Logo pink	<code>#D98A83</code>	Rose / coral	Logo only; primary button hover/pressed state
<code>bg-image</code>	Image background	<code>#F1F5F5</code>	Very light cool gray	Photo frames, avatar placeholders, upload drop zones
<code>gold</code>	Decorative gold	<code>#E9A72F</code>	Warm yellow	Decorative icons, “Verified” badge, star ratings, highlights

Usage rules:

- White text on `#E58F89` has low contrast (about 2.7:1), below the WCAG AA minimum of 4.5:1. Use `#3F312D` text on coral buttons (about 4.6:1), or a darker coral `#A94E48` (about 5.4:1) when white text is needed.
- `gold` is decorative. Do not use it for body text or warnings on cream backgrounds, where contrast is too low.
- Status colors are not in the brand palette, so add neutral-warm ones: success `#5E8C61`, warning `#C98A1B`, error `#B5473F`, info `#5B7C8D`.
- Borders and dividers: `#EADBD0` (a darkened cream) keeps the warm feel.

Tailwind config snippet for Phase 0:

```
// tailwind.config.js → theme.extend.colors
colors: {
  bg: '#FDF6EF',
  'bg-pink': '#F9E1DD',
  'bg-image': '#F1F5F5',
  primary: { DEFAULT: '#E58F89', hover: '#D98A83', strong: '#A94E48' },
  ink: { DEFAULT: '#3F312D', muted: '#826758' },
  gold: '#E9A72F',
  border: '#EADB00',
  success: '#5E8C61',
  warning: '#C98A1B',
  danger: '#B5473F',
  info: '#5B7C8D',
}
```

PRD gaps to confirm before building

The PRD contradicts itself in a few places. Each item below changes the data model or a flow, so settle it before Phase 0. The default in the right column is what this plan assumes if you don't decide otherwise.

#	Gap in the PRD	Why it matters	Default assumed in this plan
1	The intro says “Employers choose candidates and schedule interviews”; the flows say the platform runs all interviews	Decides whether families get a scheduling UI	Platform runs interviews. Families only favorite, request, and approve a recommendation
2	Contract is “between family and platform”, yet the nanny also signs it	Is it one 3-party contract or two contracts (family-platform, platform-nanny)?	One placement record with two contracts: a family service contract and a nanny work contract, each with its own signature
3	Intro allows signing “in person (printed document)”; flows cover only e-signature	Needs an offline path	Admin can download the PDF, and upload a scanned signed copy to mark it signed

#	Gap in the PRD	Why it matters	Default assumed in this plan
4	Payments: bank transfer or mobile money, but no gateway named	A live gateway (e.g. Multicaixa Express) needs merchant onboarding and can block launch	Manual recording by admin in v1; gateway integration is a later phase
5	Medical records are uploaded but visibility is not defined	Sensitive personal data	Visible to admin only, never to families
6	“Documents (if you allow families to see them)”	Privacy and UI	ID and medical hidden from families; certificates and references shown as “Verified” badges
7	Language not stated; CTAs are in Portuguese	Every screen and email	Portuguese (pt-AO) as the only UI language, with strings in a translation file so English can be added later
8	Currency not stated	Salary, pricing and reports	Angolan kwanza (AOA), whole numbers
9	Commission is “percentage or fixed fee”	Contract math	Admin chooses per contract: percentage or fixed; percentage default set in settings
10	OTP “or link” for phone and email	SMS in Angola needs a provider and costs money	Email link and SMS OTP, both sent through Brevo; WhatsApp later
11	Nanny “age” and “medical records” are collected	Data-protection law in Angola (Lei n.º 22/11) requires consent	Explicit consent checkbox at registration, logged with timestamp

Recommended stack and data model

One full-stack TypeScript app keeps the build small enough for Claude Code to hold in context. Swap any piece you already prefer; the phases do not depend on these exact choices.

Layer	Choice	Reason
App	Next.js (App Router) + TypeScript	Public pages, three dashboards and API in one repo
UI	Tailwind CSS + shadcn/ui	Palette maps straight to tokens; ready forms, tables, dialogs
Database	MongoDB + Mongoose, local MongoDB for development	Flexible profile documents; run it as a single-node replica set so transactions work for contracts and payments (Phases 5–6)
Auth	Auth.js v5 (credentials, JWT sessions) with role on the user	Email/phone + password, three roles
Validation	zod + react-hook-form	Same schemas on client and server
Files	AWS S3, private bucket, pre-signed URLs	IDs and medical records must never be public
Email	Brevo transactional email API	Verification links and notifications
SMS	Brevo transactional SMS API	Phone OTP and payment reminders
PDF	@react-pdf/renderer	Contract PDFs generated server-side
E-signature	Built in: typed name + drawn signature, IP, timestamp, document hash	Avoids a paid provider for v1
i18n	next-intl, Portuguese first	All strings outside components
Hosting	Not deployed yet; the app runs locally	Choose hosting before going live

Core collections (Mongoose models). References between collections are ObjectIds:

Model	Key fields
User	role (NANNY, FAMILY, ADMIN), fullName, email (unique, sparse), phone (unique, sparse), whatsapp, passwordHash, province, city, status (ACTIVE, SUSPENDED), emailVerifiedAt, phoneVerifiedAt, consentAt
VerificationToken	userId, type (EMAIL_VERIFY, PHONE_OTP, PASSWORD_RESET), tokenHash, expiresAt (TTL index)

Model	Key fields
FamilyProfile	userId, needDescription
NannyProfile	userId, birthDate, languages[], yearsExperience, ageGroups[] (INFANT, TODDLER, SCHOOL_AGE), skills[], employmentType, liveIn, availability (weekly grid), salaryMin, salaryMax, salaryUnit, bio, status (DRAFT, PENDING_REVIEW, APPROVED, NEEDS_CORRECTION, NOT_AVAILABLE, IN_NEGOTIATION, PLACED), verified, interviewed
NannyDocument	nannyUserId, type (PHOTO, ID, REFERENCE, CERTIFICATE, MEDICAL), s3Key, visibility (ADMIN_ONLY, FAMILY_BADGE, PUBLIC), reviewStatus, reviewNote
Favorite	familyId, nannyId (unique pair)
NannyRequest	familyId, targetNannyId (optional), childrenAges, needs, schedule, liveIn, startDate, budgetMin, budgetMax, specialRequirements, status (NEW, MATCHING, INTERVIEWING, PROPOSED, APPROVED, CONTRACTED, CLOSED)
RequestCandidate	requestId, nannyId, contactStatus (CONTACTED, INTERESTED, NOT_INTERESTED), availabilityConfirmed, isRecommended
Interview	candidateId, scheduledAt, mode (IN_PERSON, PHONE, VIDEO), status (SCHEDULED, DONE, NO_SHOW, CANCELLED), outcome, notes, score
Placement	requestId, familyId, nannyId, startDate, endDate, status (ACTIVE, ENDED, TERMINATED, REPLACED)
Contract	placementId, party (FAMILY, NANNY), version, terms, nannySalary, familyTotal, commissionType, commissionValue, commissionAmount, status (DRAFT, SENT, SIGNED, ACTIVE, ENDED, TERMINATED), pdfKey, signedPdfKey, docHash
Signature	contractId, userId, typedName, imageKey, ip, userAgent, signedAt, method (ONLINE, IN_PERSON)

Model	Key fields
Payment	placementId, direction (IN_FROM_FAMILY, OUT_TO_NANNY), periodMonth, amount, method, reference, paidAt, recordedById
Notification	userId, type, title, body, link, readAt, channelsSent (IN_APP, EMAIL, SMS)
SupportTicket	openedById, placementId (optional), type (ISSUE, COMPLAINT, REPLACEMENT), status, resolution, messages[]
AuditLog	actorId, action, entity, entityId, before, after, createdAt

The family's contract amount is `familyTotal = nannySalary + commissionAmount`. For a percentage commission, `commissionAmount = nannySalary × rate`. Store all amounts as integers in kwanza.

Phase 0 — Foundation and project setup

Goal: an empty but fully wired app that runs locally and deploys, with the design system, database schema and translations in place.

Build:

- ☐ Next.js + TypeScript project, ESLint, Prettier, `.env.example` (MongoDB URI, AWS S3 keys and bucket, Brevo API key and sender)
- ☐ Tailwind with the palette tokens from the Color palette section; base font, spacing, radius
- ☐ shadcn/ui installed; restyle Button, Input, Select, Card, Badge, Table, Dialog, Tabs to the palette
- ☐ MongoDB connection helper (cached on `globalThis`), Mongoose models for every collection in the data model section, indexes
- ☐ Idempotent seed script: 1 admin, 5 nannies, 3 families
- ☐ Angola location constants: all 18 provinces with their main cities, used by every province/city dropdown
- ☐ `next-intl` with a `pt` messages file; no hard-coded UI strings
- ☐ Layout shells: public layout (header with logo, “Sou Babá” / “Sou Família”, footer), dashboard layout with sidebar per role

- ☐ `StorageService` on AWS S3: upload to private bucket, pre-signed download URL, delete
- ☐ `EmailService` and `SmsProvider` on the Brevo API, with a dev flag that logs instead of sending
- ☐ Audit log helper called from every admin write

Done when:

- `npm run dev` shows the public shell in the palette colors
- The app connects to local MongoDB and the seed runs twice without duplicates
- A test file uploads to the S3 bucket and opens only through a pre-signed URL
- One test email and one test SMS arrive through Brevo
- Nothing is deployed; the app runs locally only

Phase 1 — Authentication and onboarding

Goal: nannies and families can register, verify, log in and land on their own dashboard; admins log in to theirs.

Build:

- ☐ “Sou Babá / Registrar” form: full name, email and/or phone, password, city, province, phone/WhatsApp, consent checkbox
- ☐ “Sou Família / Registrar” form: same fields plus one free-text description of need (`needDescription`)
- ☐ Province and city dropdowns from the Phase 0 seed list of all 18 Angolan provinces and their cities; city options filter by the chosen province
- ☐ Email verification link sent via Brevo email; phone OTP sent via Brevo SMS
- ☐ Login, logout, forgot password, reset password
- ☐ Role-based route protection: `/baba/*`, `/familia/*`, `/admin/*`; wrong role gets redirected
- ☐ Suspended accounts cannot log in
- ☐ Empty dashboards per role showing the user’s name and next step (“Complete o seu perfil”)
- ☐ Account settings: edit contact details, change password
- ☐ Rate limiting on login, register and OTP endpoints

Done when:

- A new nanny and a new family can each register, verify and reach their dashboard
- A family cannot open any `/baba` or `/admin` page, even by URL

- The seeded admin reaches `/admin`

Phase 2 — Nanny profile and document verification

Goal: a nanny builds a complete profile with documents, submits it, and an admin approves or sends it back for correction.

Build (nanny side):

- ☐ Multi-step profile editor with a progress bar: Personal, Experience, Availability, Documents, Review
- ☐ Personal: birth date (age computed), languages, location
- ☐ Experience: years, age groups (infants, toddlers, school-age), skills (first aid, cooking, homework help, special needs, other)
- ☐ Availability: full-time/part-time, live-in/live-out, weekly day-and-hour grid, salary expectation (monthly or hourly, AOA)
- ☐ Uploads: profile photo (cropped square), ID document, optional references and certificates, medical records; size and type limits
- ☐ “Submit for review” moves status to PENDING_REVIEW and locks key fields
- ☐ Dashboard: profile completion %, current status, correction notes from admin

Build (admin side):

- ☐ Nannies list with filters by status, province, verified
- ☐ Nanny detail: all fields and documents via signed URLs
- ☐ Actions: Approve, Request correction (with note per document or field), Mark verified (ID checked), Suspend
- ☐ Manual status changes: Not available, In negotiation, Placed

Profile status flow:

```
stateDiagram-v2
    [*] --> Draft
    Draft --> PendingReview: nanny submits
    PendingReview --> Approved: admin approves
    PendingReview --> NeedsCorrection: admin requests fix
    NeedsCorrection --> PendingReview: nanny resubmits
    Approved --> InNegotiation: matched to request
    InNegotiation --> Placed: contract active
    InNegotiation --> Approved: not selected
    Placed --> Approved: placement ends
    Approved --> NotAvailable: nanny pauses
    NotAvailable --> Approved: nanny resumes
```

Only APPROVED profiles are visible to families. Editing an approved profile's key fields sends it back to review.

Done when:

- A seeded nanny completes all steps, submits, gets a correction note, fixes it and is approved
- Medical and ID files cannot be opened by any family account

Phase 3 — Family search, favorites and requests

Goal: a family finds approved nannies, saves favorites and submits a request that lands in the admin queue.

Build:

- ☐ Search page with filters: province/city, experience level, full-time/part-time, live-in/live-out, age groups, languages, salary range
- ☐ Result cards: photo, first name + initial, city, years of experience, top skills, salary range, "Verified" gold badge
- ☐ Pagination and sort (newest, experience, salary)
- ☐ Public nanny profile page for families: summary, experience, skills, availability, badges; no contact details, no ID or medical files
- ☐ Favorite / unfavorite; "Shortlisted nannies" list on the family dashboard
- ☐ "Request this nanny" button (request with `targetNannyId`) and a general "Request a nanny" form
- ☐ Request form fields: children's ages and needs, days/hours, live-in or live-out, start date, budget range (AOA), special requirements
- ☐ Family dashboard: active requests with status (New, Matching, Interviewing, Proposed, Approved, Contracted)

No nanny contact details are shown to families anywhere. All contact goes through the platform, as the PRD requires.

Done when:

- Filters return correct seeded nannies for 3 test combinations
- A family favorites 2 nannies, requests 1, and sees the request as "New"
- The request appears in the admin database (the admin screen comes in Phase 4)

Phase 4 — Admin panel, matching and interviews

Goal: the admin turns a family request into a recommended nanny that the family approves, all inside the platform.

Build:

- ☐ Admin dashboard with KPI cards: total nannies, total families, open requests, active contracts, revenue this month (revenue fills in Phase 6)
- ☐ Families management: list, detail, suspend
- ☐ Requests queue: list by status, request detail with the family's requirements
- ☐ Matching panel inside a request: nanny search with the request's filters pre-applied; "Add as candidate"
- ☐ Candidate tracking per request: Contacted, Interested / Not interested, Availability confirmed
- ☐ Nanny side: "Interview invitations" section showing the family's situation (no family contact details) with Accept / Decline
- ☐ Interviews: schedule (date, time, mode), reschedule, record outcome, notes, score; mark nanny as "Interviewed"
- ☐ "Recommend to family": pick 1-3 candidates with profile summary, interview notes and a recommendation text
- ☐ Family side: "Recommended nannies" view on the request; Approve one or Ask for other options
- ☐ On approval: request → APPROVED, chosen nanny → IN_NEGOTIATION, other candidates released

Matching flow:

```

flowchart LR
    A[Family request] --> B[Admin shortlists]
    B --> C[Nanny accepts invite]
    C --> D[Platform interview]
    D --> E[Recommend to family]
    E --> F{Family decision}
    F -->|Approve| G[Contract - Phase 5]
    F -->|Other options| B
  
```

Done when:

- One request goes end to end: shortlist 3, one declines, interview 2, recommend 2, family approves 1
- The rejected and declined nannies return to APPROVED status

Phase 5 — Contracts and e-signature

Goal: from an approved match, the admin issues contracts that the family and nanny sign online (or in person), producing downloadable signed PDFs and an active placement.

Build:

- ☐ Contract builder from an approved request: nanny, start date, schedule, duties, nanny salary, commission (percentage or fixed), notice period, termination terms
- ☐ Auto-calculation: commission amount and family monthly total, shown live in the builder
- ☐ Two contract templates in Portuguese: Family service contract (shows total monthly cost, commission, payment terms, platform as employer/mediator) and Nanny work contract (role, duties, hours, salary, payment schedule, notice)
- ☐ PDF generation with the logo and palette; document hash stored at send time
- ☐ Send for signature: status DRAFT → SENT; notification to the party
- ☐ Signing screen: read contract, typed full name, drawn signature, “I agree” checkbox; store IP, user agent, timestamp
- ☐ Signed PDF regenerated with a signature page and stored
- ☐ In-person path: admin downloads PDF, uploads scanned signed copy, marks signature method IN_PERSON
- ☐ Contracts section for nanny and family: current contract, history, download PDF
- ☐ Admin contracts list with status filter; terminate or end with a reason
- ☐ When both are signed: contracts → ACTIVE, placement → ACTIVE, nanny → PLACED, request → CONTRACTED

Contract status flow:

```
stateDiagram-v2
    [*] --> Draft
    Draft --> Sent: admin sends
    Sent --> Signed: party signs
    Signed --> Active: both contracts signed
    Active --> Ended: end date reached
    Active --> Terminated: early termination
```

The PRD’s combined statuses (“Signed by family”, “Signed by nanny”) show on the admin placement view, one per contract.

Done when:

- The approved match from Phase 4 becomes an active placement with two signed PDFs
- Editing a sent contract creates a new version and voids the old signature request

- The in-person path works with an uploaded scan

Phase 6 — Payments, commissions and reports

Goal: the admin records money in from families and out to nannies each month, balances and commission compute themselves, and both users see their payment history.

Build:

- ☐ Monthly billing schedule per active placement: one expected family payment and one expected nanny payout per month
- ☐ Record payment received from family: month, amount, method (bank transfer, Multicaixa Express, mobile money, cash), reference, date, optional receipt upload
- ☐ Record payment made to nanny: same fields
- ☐ Auto-calculated per placement: commission earned, family outstanding balance, nanny outstanding balance
- ☐ Overdue flag when a family payment is past the contract due day
- ☐ Family “Payments” section: amounts due, paid, receipts (PDF)
- ☐ Nanny “Payments” section: scheduled and completed payouts
- ☐ Reports: monthly commission revenue, total paid to nannies, active contracts count, overdue list; filter by month; CSV export
- ☐ Dashboard revenue KPI now filled

Payment gateway integration stays out of this phase. Keep the `Payment` model gateway-ready (method + external reference) so it can be added later without schema changes.

Done when:

- For one placement across 2 months, recorded amounts give the right commission and zero balances
- The monthly report matches a hand calculation
- A family receipt PDF downloads

Phase 7 — Notifications, support and disputes

Goal: every event in the PRD’s notification list reaches the right person in-app and by email, and families and nannies can raise issues that the admin resolves.

Build:

- ☐ One `notify(userId, type, data)` service: writes the in-app notification and sends email and SMS through Brevo; WhatsApp channel later
- ☐ Bell icon with unread count and a notifications page

- ☐ Portuguese email templates styled with the palette
- ☐ Hook `notify` into the events in the table below
- ☐ Support tickets: families and nannies open a ticket (Issue, Complaint, Replacement request), optionally linked to a placement; message thread; status Open / In progress / Resolved
- ☐ Admin tickets queue: assign, reply, resolve with a resolution note
- ☐ Replacement flow: resolving a replacement ticket opens a new request prefilled from the old one, and the old placement ends as REPLACED

Recipient	Event	Triggered in
Nanny	Profile approved / needs correction	Phase 2
Nanny	New interview invitation; interview confirmed / rescheduled	Phase 4
Nanny	Contract ready to review; contract signed	Phase 5
Nanny	Payment scheduled / completed	Phase 6
Family	Registration confirmed	Phase 1
Family	New nanny recommendations; process update (“We found 3 suitable nannies”)	Phase 4
Family	Contract ready to review; signed by all parties	Phase 5
Family	Payment reminder / receipt	Phase 6
Admin	New nanny submitted for review; new request; new ticket	Phases 2–4, 7

Payment reminders run on a daily scheduled job (node-cron worker), 3 days before and on the due date.

Done when:

- Running the full Phase 4–6 scenario produces every notification in the table
- A replacement ticket produces a new request with the family’s original requirements

Phase 8 — Public pages, polish, QA and launch

Goal: the public site explains the service and converts visitors, the whole app works on phones, and it is ready to deploy when you decide to go live.

Build:

- ☐ Homepage: headline “Ligamos famílias a babás de confiança e tratamos de tudo por si”, “Sou Babá” / “Sou Família” CTAs, How it works in 4 steps, trust section (verified, interviewed, contracted), blush-pink alternating sections
- ☐ Pages: How it works, About us, Contact / Support form, Terms, Privacy Policy (legal text supplied by you or your lawyer)
- ☐ Mobile pass on every screen at 360 px width; most Angolan users will be on phones
- ☐ Empty, loading and error states on all lists and forms
- ☐ Accessibility pass: contrast per the palette rules, labels on inputs, keyboard focus visible
- ☐ Security pass: role checks on every server action and API route, signed URLs expire, file-type validation, no nanny contact data in family responses
- ☐ SEO basics: titles, meta descriptions, Open Graph image, sitemap
- ☐ End-to-end tests (Playwright) for: nanny registration → approval; family request → recommendation → approval; contract signing; payment recording
- ☐ Deployment checklist only, no deploy yet: hosting choice, domain, HTTPS, production MongoDB with backups, production S3 bucket and Brevo sender, error tracking (Sentry), admin account, seed data removed

Done when:

- All four end-to-end tests pass against a local production build
- A real phone completes nanny and family registration
- The deployment checklist is complete; going live is a separate decision

Working with Claude Code

Run one phase per Claude Code session: plan first, approve the plan, build, test against the phase’s “Done when” list, commit. Keep a `CLAUDE.md` at the repo root so every session starts with the same rules.

Put this in `CLAUDE.md`:

```
# Nanny Platform (Angola)

## Product
Platform is the official employer/mediator between families and nannies.
Families never get nanny contact details. Platform runs matching, interviews,
contracts and payments. Roles: NANNY, FAMILY, ADMIN.

## Stack
Next.js App Router + TypeScript, Tailwind + shadcn/ui, MongoDB + Mongoose
(local),
```

Auth.js, next-intl (Portuguese pt-AO), private AWS S3 bucket with pre-signed URLs, Brevo API for email and SMS, @react-pdf/renderer for contracts.

Rules

- UI text only via next-intl messages (pt). No hard-coded strings.
- Colors only via Tailwind tokens (bg, bg-pink, bg-image, primary, ink, gold). No hex in components.
- Money: integer kwanza (AOA). familyTotal = nannySalary + commissionAmount.
- Every server action and API route checks role. Admin writes call auditLog().
- ID and MEDICAL documents are ADMIN_ONLY. Never return them to FAMILY.
- Never expose nanny email/phone/WhatsApp to FAMILY.
- Local development only. Do not add deployment config.
- Age groups: INFANT, TODDLER, SCHOOL_AGE. FamilyProfile holds only needDescription.
- Province/city dropdowns use the 18-province seed list in constants.
- Before coding a phase: write a plan, list files to create/change, wait for approval.
- After a phase: run lint, typecheck, tests; list what is done vs the phase checklist.

Palette

bg #FDF6EF | bg-pink #F9E1DD | primary #E58F89 | primary-hover #D98A83
 primary-strong #A94E48 | ink #3F312D | ink-muted #826758 | bg-image #F1F5F5 |
 gold #E9A72F

Phase kickoff prompt (paste into Claude Code, replacing the phase number and pasting that phase's section):

We are starting Phase N of the nanny platform. Read CLAUDE.md first.

Here is the phase scope and its "Done when" list:
 <paste the Phase N section from the plan doc>

Step 1: Inspect the current repo and give me a plan: files to add/change, schema changes (if any), and anything in this scope that conflicts with what already exists. Do not write code yet.

Step 2: After I approve, build it in small commits.

Step 3: Seed data for testing this phase, then walk through every "Done when" item and report pass/fail with evidence.

Habits that keep the build on track:

- Commit and tag at the end of each phase (phase-0-done , phase-1-done ...) so you can roll back a bad session

- Start a fresh Claude Code session per phase; long sessions lose earlier context
- If a phase spills over, cut scope from the “Build” list rather than skip its “Done when” checks
- Keep the PRD gaps table updated as you decide each item, and paste the decision into `CLAUDE.md`